

```

#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>
using namespace std;

int main() {
    int n;
    cout << "Enter number of vertices: ";
    cin >> n;

    vector<vector<int>> graph(n, vector<int>(n));

    cout << "Enter adjacency matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];
        }
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;

    vector<bool> visited(n, false);
    stack<int> st;
    st.push(start);

    cout << "DFS Traversal: ";

    while (!st.empty()) {
        int node;

        // Safe pop from stack
        #pragma omp critical
        {
            node = st.top();
            st.pop();
        }

        if (!visited[node]) {
            visited[node] = true;
            cout << node << " ";

            // Keep DFS logic correct (no parallel here)
            for (int i = n - 1; i >= 0; i--) {
                if (graph[node][i] == 1 && !visited[i]) {
                    st.push(i);
                }
            }
        }
    }

    cout << endl;
}

```

```
    return 0;  
}
```